

Simulating FM Radio Transmissions in Max/MSP

Henrik Forssell

Independent Researcher

henrik.forssell11@gmail.com

ABSTRACT

This paper presents a simulation of frequency modulation (FM) radio tuning effects—including distortion, static noise, and cross-fading between stations—implemented in the Max/MSP programming environment. By recreating the modulation and demodulation stages of discrete-time FM, the system allows users to simulate tuning across multiple radio stations using their own audio source material. The implementation, titled `radiofm`, is released as open-source and includes Max abstractions for FM modulation, demodulation, channel behavior, and an example patch. The paper describes the underlying DSP implementation of the Max abstractions and provides a perceptual evaluation of the resulting tuning behavior. Potential use-cases and limitations are discussed, illustrating how the `radiofm` modules can be used both as artistic and experimental DSP tools.

1. INTRODUCTION

Since its invention in 1933, Frequency modulation (FM) broadcasting has had a major impact on society both as an information and cultural technology and wide-band FM is still used around the world to transmit audio signals over radio broadcasts. The sound of FM radio has been used in both interactive art and modern music technology, particularly within the field of radio art [1]. Examples within music technology include the Eurorack module Radio Music that plays samples from an SD card and integrates a radio-style cross-fading between stations into a Eurorack system. Another example is Teenage Engineering’s OP-1 that includes a real FM radio receiver that allows the user to sample directly from over-the-air FM transmissions. In addition, various effect presets exist for simulating the audio effects of a “bad radio signal”. Undoubtedly, with the advent of the internet and web-based information technologies, new mediums for consuming information and music arise and radio broadcast systems have become somewhat functionally outdated.

Recreating the sound of *tuning between FM radio stations* can have many artistic applications. While some can be achieved through sampling techniques (i.e., recording real-world radio signals), others may require artists to apply the effect to their own audio source material. Creating the perception of tuning between radio stations could

be realized through a combination of well-known audio processing techniques such as equalization, distortion, and noise. However, for the purposes of fidelity and realism we can consider simulating the process of FM modulation and demodulation at audio DSP rate (i.e., without ever transmitting signals over-the-air).

To this end, this paper presents the implementation of an FM radio simulator inside the visual programming environment Max/MSP [2]. The developed implementation is titled `radiofm` and consists of Max abstractions for modulating and demodulating FM signals inside Max, creating multiple “radio stations”, and simulating wireless channel fading effects. Possible use-cases of `radiofm` include generative music with simulated tuning between virtual radio stations and sending FM modulated audio streams between separate Max applications. The `radiofm` abstractions are made available as open-source software [3] including an example patch that demonstrates how the modules can be interconnected to simulate a complete FM radio transmission system.

2. RELATED WORK

Demodulation of FM radio has previously been implemented in Max/MSP as part of projects integrating software defined radios (SDRs) [4] and web-based SDRs like Open-WebRx and KiwiSDR [5]. These works focus on demodulating real over-the-air radio signals using SDRs connected to Max rather than simulating the effect of FM radio tuning within Max. There are also a few informal discussions on the topic of simulating the sound of FM radio on the Max Forum (see e.g., the thread “Radio bad signal effect?” from 2011 [6]). Many interesting and creative Max patches are provided (e.g., noisy reading of audio buffers and use of ring modulation), however, none of the patches appear to include a detailed simulation of the FM modulation and demodulation process.

Within interactive art and music there are several examples of use of FM radio technology. Perhaps most commonly known is John Cage’s *Imaginary Landscape No. 4* which is a composition for 24 performers and 12 radios. In the exhibition *Exaudimus* [7] from 2013, aimed at preserving the cultural heritage of Danish radio, visitors walked around the installation space wearing headsets that acted like old FM receivers tuning between stations depending on the location of the visitor. According to the associated publication, Max was used for the implementation and static noise was sent to the headsets when a visitor was located “between stations”. Another example is found in the audio installation works of Anna Friz, where a sonic soundscape is created based on 12-25 FM receivers and multiple FM micro-transmitters [1].

3. FM RADIO PRELIMINARIES

In FM transmissions, the continuous-time modulated signal is given by

$$y(t) = A \sin \left(2\pi f_c t + 2\pi k_f \int_0^t x(\tau) d\tau \right), \quad (1)$$

where the encoded information signal is denoted by $x(t)$, f_c is the carrier frequency, and k_f is the frequency deviation constant. The FM modulated signal $y(t)$ is transmitted over some medium (typically electromagnetic radio waves) and a noisy version is received at another point in space. Demodulation is the process of recovering the information signal $x(t)$ from the noisy version of $y(t)$ and typically consists of (i) accessing the instantaneous phase $\phi(t) = 2\pi f_c t + 2\pi k_f \int_0^t x(\tau) d\tau$, (ii) differentiating the phase to obtain the instantaneous frequency $2\pi(f_c + k_f x(t))$, and (iii) subtracting the carrier frequency and dividing by $2\pi k_f$ so that

$$x(t) = \frac{1}{2\pi k_f} \left(\frac{d\phi(t)}{dt} - 2\pi f_c \right). \quad (2)$$

In practice, analog implementations are based on frequency discriminators that convert frequency deviations to amplitude changes, or phase locked loops (PLLs).

The software modules presented in the following Section 4 consider a discrete-time implementation of the analog processes in Equation (1) and (2) that can be entirely simulated inside Max/MSP. The transmission "medium" for $y(t)$ in the simulation is audio-rate signals bandlimited by the sample rate setting on the host computer and Max/MSP application.

4. IMPLEMENTATION IN MAX/MSP

The `radiofm` modules consist of Max abstractions for FM modulation, demodulation, and a channel simulator for modeling channel fading and noise. The example patch `radiofm-example.maxpat` demonstrates how the FM modulation, demodulation, and channel simulator modules can be interconnected to simulate a complete FM radio transmission system. An overview of the example patch can be seen in Figure 1. The patch includes:

- Three signal sources (i.e., radio stations) consisting of `playlist~` objects looping different audio samples. These can be replaced by arbitrary audio sources generated in Max or by external inputs.
- The FM modulator abstraction (`radiofm.mod`) generates the modulated signals. This abstraction has frequency deviation k_f as first argument and allows signal inputs for audio signal, carrier frequency, and input gain.
- The channel abstraction (`radiofm.channel`) combines the modulated signals and adds noise and fading effects. This abstraction accepts inputs for amount of fading and noise floor level.
- The FM demodulator abstraction (`radiofm.demod`) recovers the signal at a given carrier frequency and plays it as an audio signal in this case connected to

an `ezdac~`. This abstraction has two signal inlets, one for the modulated signal and one for the carrier frequency.

Note that since the DSP implementation of FM modulation involves complex-valued signals with in-phase (I) and quadrature (Q) components, the modulated signals in the example patch are carried by blue multi-channel (mc) patch-cords. The following sections will describe the detailed implementation of the `radiofm` abstractions.

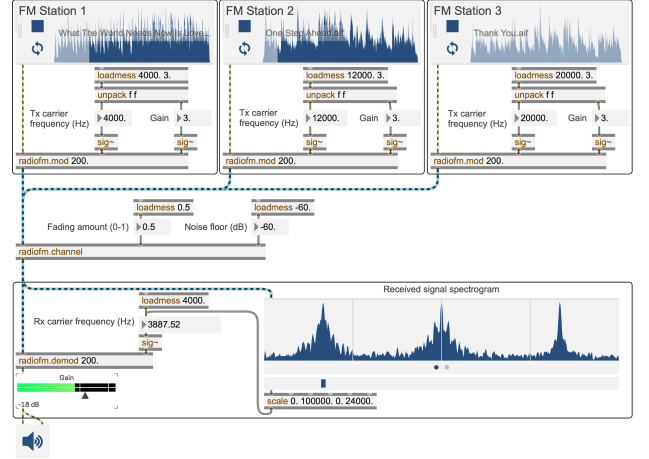


Figure 1. Provided example patch `radiofm-example.maxpat` with three FM modulated carriers, channel, and demodulator.

4.1 FM Modulator

In discrete time, the FM modulation in (1) can be implemented by computing the instantaneous phase according to the difference equation

$$\phi[n] = \phi[n-1] + \frac{2\pi k_f}{F_s} x[n], \quad (3)$$

where $x[n]$ is the discrete-time information signal and F_s denotes the sample rate. The `gen~` patch for FM modulation can be seen in Figure 2 where the instantaneous phase in (3) is used to phase modulate a complex oscillator with frequency f_c . As can be seen in Figure 2, the `history` object is used for the one-sample delay and the `phasor` object is well suited for generating the carrier frequency.

In addition to the main `gen~` patch, the `radiofm.mod` abstraction has a pre-processing stage consisting of a band-limiting lowpass filter based on the `cascade~` and `filterdesign` objects, a gain stage, and a limiter based on `clip~` that will limit the frequency modulation to $\pm k_f$.

4.2 FM Demodulator

The `radiofm.demod` abstraction is implemented as a three stage process consisting of (i) removing the carrier frequency, (ii) differentiating, and (iii) recovering the phase. The `gen~` implementation of these stages can be seen in Figure 3. Removing the carrier is achieved by a multiplication with a conjugate complex oscillator

$$z[n] = y[n] e^{-j2\pi \frac{f_c}{F_s} n}, \quad (4)$$

where we denote the received complex-valued signal by $y[n]$. In the ideal case this recovers the instantaneous phase

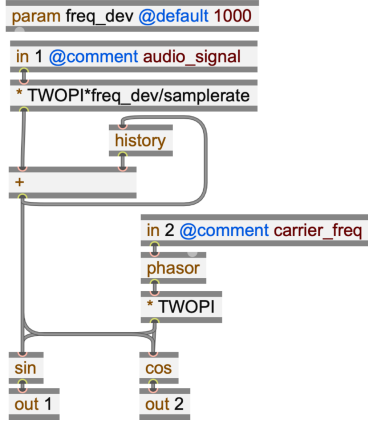


Figure 2. FM modulation implementation in gen~.

of the modulated signal, i.e., $z[n] = e^{j\phi[n]}$. The differentiation step is performed by defining

$$c[n] = z[n] * z^*[n-1] = e^{j(\phi[n] - \phi[n-1])}, \quad (5)$$

i.e., multiplying $z[n]$ with a delayed and conjugate version of itself. Finally, seeing that $\arg(c[n]) = \phi[n] - \phi[n-1] = \frac{2\pi k_f}{F_s} x[n]$, we can recover the information signal by

$$x[n] = \frac{F_s}{2\pi k_f} \arg(c[n]), \quad (6)$$

where $\arg(c[n])$ can be computed by the `atan2` object inside `gen~`. As a final stage of the `gen~` implementation there is a `dcblock` object for removing any DC signal component in the demodulated signal.

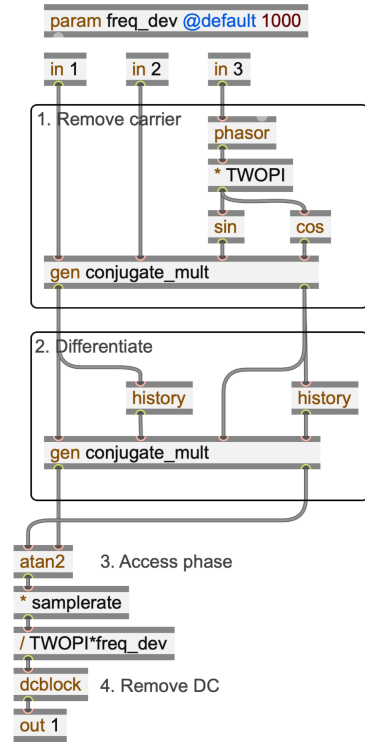


Figure 3. FM demodulation implementation in gen~.

In addition to the main demodulation steps illustrated in

the `gen~` patch in Figure 3, the `radiofm.demod` abstraction also has a bandpass filter centered around the receiver carrier frequency implemented as two `cascade~` filters and a final mixing stage consisting of another band-limiting `cascade~` filter and a `limi~` limiter.

4.3 Channel Simulator

The `radiofm.channel` abstraction takes as input a 2 channel (I/Q) modulated signal and outputs a simulated received signal with fading and noise. The output of the channel is modelled as follows

$$y_{ch}[n] = a[n] * x_{ch}[n] + w[n], \quad (7)$$

where $x_{ch}[n]$ is the complex-valued input signal, $a[n]$ is the channel fading, and $w[n]$ is additive white Gaussian noise (AWGN). The implemented abstraction can be seen in Figure 4, where the fading process $a[n]$ is implemented as a lowpass-filtered Rayleigh random process where the `onepole~` is used for the filtering, i.e., low center-frequency means $a[n] \approx 1$ and higher center-frequency means that $a[n]$ will fluctuate more rapidly. The random processes are implemented inside `gen~` where the `noise` object is used to generate uniformly distributed random variables and a Box-Muller transform is used to create Gaussian and Rayleigh distributions, respectively.

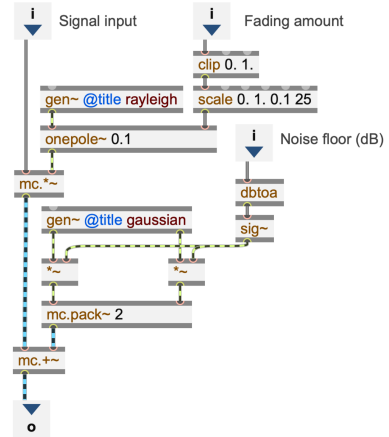


Figure 4. Channel simulator abstraction introducing Rayleigh fading and AWGN noise.

5. PERCEPTUAL EVALUATION AND DISCUSSION

The implementation has been tested and evaluated using a variety of music and speech source materials. The evaluations were conducted on a MacBook Pro M2 running Max 9.0.5, with a DSP sample rate of 48 kHz and a signal vector size of 64. The tests in `radiofm-example.maxpat` involve modulating three carriers at 4kHz, 12kHz, and 20kHz, respectively, with frequency deviation $k_f = 200$ Hz, and band-limiting filters set to 8 kHz. The resulting output have several characteristics that are typical of FM radio tuning, such as signal distortions, phase noise, and the perceptual effect of hearing a mix of two stations when the receiver frequency falls between two carriers. Additionally, the band-limiting filters with a cutoff frequency of 8 kHz contribute a lo-fi character to the sound.

The distortions and phase noise do have a distinctly digital character and do not precisely capture the behavior of older analog FM radios. This is expected since the DSP implementation presented in this paper cannot capture the nonlinearities and colorations of analog circuitry present in vintage FM radios.

5.1 Use Cases

The `radiofm` abstractions open up a range of use-cases beyond the provided example patch. First, since the simulation is entirely contained within Max, it allows for complex modulation of system parameters such as carrier frequencies and noise levels. This makes it possible to create generative music pieces that automatically or randomly tune between virtual radio stations according to complex modulation patterns. Moreover, integration with Max for Live devices can enable the use of FM radio-style effects within Ableton Live. Additionally, note that since the modulated signals are normal MSP multi-channel signals inside Max, it is straightforward to process these signals in other ways than the provided `radiofm.channel` (like e.g., running the modulated signal through a multi-tap delay line to create novel chorusing or interference textures). This may enable creative sound design techniques that stretch beyond the typical sound of FM radio tuning. Another interesting extension involves using an audio interface and stereo cable to transmit the modulated multi-channel signal through a physical channel. Such a setup allows one Max instance to act as a transmitter while another receives and demodulates the signal, enabling more sophisticated system architectures.

5.2 Limitations and Future Work

One limitation of the implementation is the restricted bandwidth of the audio spectrum imposed by the DSP sample rate. In contrast to the FM radio band in the electromagnetic spectrum (87.5–108.0 MHz), MSP running at 48 kHz offers a much narrower range. This constrains the number of stations that can be simulated without significant inter-channel interference. According to Carson’s rule, the bandwidth of an FM carrier is approximately $B_{\text{FM}} \approx 2(k_f + B)$ where B is the bandwidth of the modulating signal. In the provided example patch, this results in a bandwidth of approximately 16.4 kHz per carrier, which already pushes the limits of the available spectrum. This limitation can be partly alleviated by running Max at a higher sampling rate of 96 kHz, effectively doubling the available spectrum at a cost of higher CPU load. Another perceptual workaround is to limit the simulation to two carriers and dynamically switch between different information signals to create the illusion of tuning through a greater number of stations.

Another limitation is that the current implementation only supports modulation of mono audio signals. Real FM radio supports stereo by using sum and difference signals and modulating double-sideband suppressed carrier (DSB-SC) signals. While an updated implementation including stereo FM modulation is left for future work, a perceptual workaround is possible by letting two parallel carriers simulate modulation and demodulation of the left and right stereo channels.

The perceptual evaluation presented in this paper is limited to informal listening tests performed by the author. Controlled evaluation with side-by-side comparisons to other simulation methods and real FM radio recordings is also left for future work.

Finally, the `radiofm` abstractions are released as open-source so users can build custom interfaces or integrate the objects into existing Max patches. A dedicated Max for Live device built on the same core is in development and will be released separately.

6. CONCLUSIONS

This paper has presented the `radiofm` FM radio simulator in Max MSP consisting of abstractions for FM modulation, demodulation, and wireless channel simulation. The paper showed the DSP architecture of the implemented modules and the example patch demonstrating how the modules can be interconnected to simulate a complete FM radio transmission system, all available as open-source Max patches and abstractions. A perceptual evaluation revealed that the resulting sound possesses many of the typical characteristics of FM radio tuning, while the introduced static noise between stations had a more digital sound when compared to analog FM radio receivers. Possible use-cases include generative music with simulated tuning between virtual radio stations, sending FM modulated audio streams between separate Max applications, and future implementations of FM radio effects in Max for Live.

7. REFERENCES

- [1] C. Black, “An Overview of Spatialised Broadcasting Experiments with a Focus on Radio Art Practices,” *Organised Sound*, vol. 15, no. 3, pp. 198–208, 2010.
- [2] Cycling ’74, “Max 9,” <https://cycling74.com/products/max>, 2025, visual programming environment for music and multimedia.
- [3] H. Forssell, “FM Radio Simulator for MaxMSP (`radiofm`),” <https://github.com/hefo/maxmsp/tree/main/radiofm>, 2025, accessed: 2025-05-16.
- [4] D. Slater, “Software Defined Radio in Max/MSP,” <http://www.zerokidz.com/radio/docs/maxsdr5.pdf>, 2013, technical documentation for a software-defined radio receiver built in Max/MSP.
- [5] T. Zicarelli, “OpenWebRx and KiwiSDR added to Max/MSP control program,” <https://reactivemusic.net/?p=20971>, 2024, blog post on reactiveMusic.net describing updates to a Max/MSP remote receiver project.
- [6] Cycling ’74 Forum Users, “Radio Bad Signal Effect,” <https://cycling74.com/forums/radio-bad-signal-effect>, 2011, Max/MSP Forum discussion thread.
- [7] C. H. Mortensen and V. Vestergaard, “Embodied Tuning: Interfacing Danish Radio Heritage,” *Journal of Interactive Humanities*, vol. 1, no. 1, 2013. [Online]. Available: <https://repository.rit.edu/jih/vol1/iss1/3>